Lập trình mạng · Chương 4

gRPC: Protocol Buffers và lời gọi unary

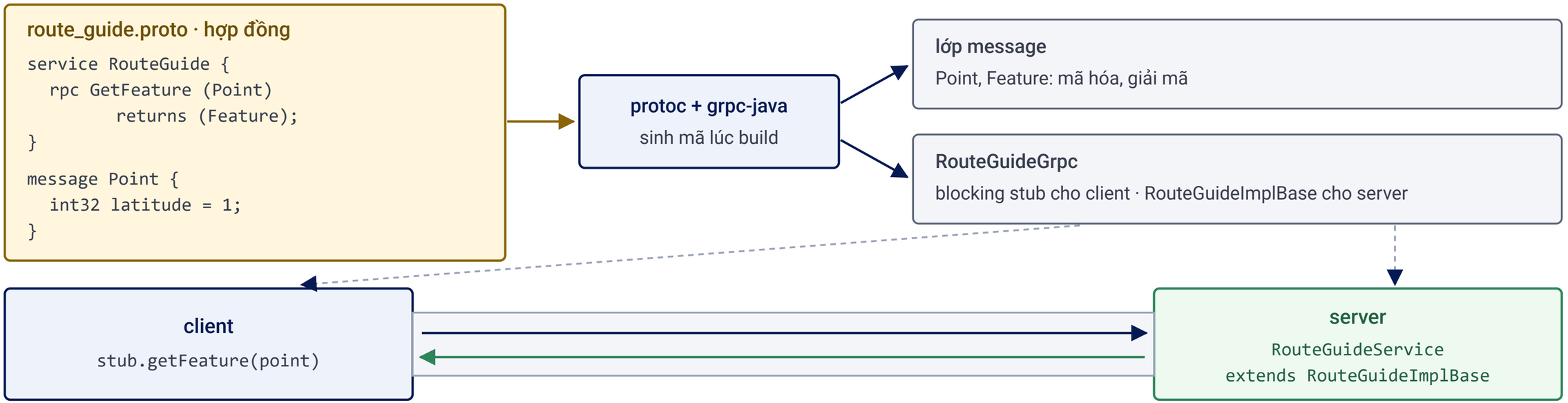
TS. Đặng Ngọc Hùng

gRPC: quay lại RPC với hợp đồng chặt và message nhị phân

1. REST + JSON: hợp đồng **không bắt buộc**; gõ sai tên trường chỉ lộ ra lúc chạy
2. JSON là văn bản, mỗi message mang cả **tên trường**
3. **gRPC**: hợp đồng bằng **Protocol Buffers**, message nhị phân, chạy trên **HTTP/2**

> **Câu hỏi của phần:** định nghĩa dịch vụ bằng file .proto, sinh mã và gọi một RPC thế nào?

Từ file .proto đến lời gọi: protoc sinh lớp message và stub



một kết nối HTTP/2, nhiều lời gọi song song (multiplexing)

Sinh mã: từ **file .proto**, protoc tạo lớp message và stub; client và server dùng chung một hợp đồng.

Protocol Buffers mã hóa bằng số trường: 11 byte so với 45 byte JSON

Point { latitude: 209809000, longitude: 1057871000 }



Protocol Buffers: 11 byte

08: trường 1, 10: trường 2, kiểu varint: số nguyên mã hóa trong 1–5 byte

JSON cùng nội dung

```
{"latitude":209809000,"longitude":1057871000}
```

45 byte

tên trường và từng chữ số đi trên mạng

kích thước đo được	Protocol Buffers	JSON
Point	**11 byte**	45 byte
Feature (tên + Point)	**66 byte**	119 byte

so sánh	REST + JSON	gRPC + Protocol Buffers
hợp đồng	không bắt buộc (OpenAPI)	bắt buộc: file .proto, sinh mã
giao thức	HTTP/1.1 hoặc HTTP/2	HTTP/2
trình duyệt	gọi trực tiếp	cần gRPC-Web

> **Protocol Buffers**: mỗi trường gồm tag (số trường, kiểu) rồi giá trị; số nguyên mã hóa varint; tên trường không đi trên mạng.

File .proto: dịch vụ, RPC unary và message có số trường

```protobuf
service RouteGuide {
  rpc GetFeature(Point) returns (Feature) {}  // unary: một request, một response
}

message Point {
  int32 latitude = 1;  // số trường 1: định danh trường khi mã hóa, không được đổi
  int32 longitude = 2;
}
message Feature {
  string name = 1;
  Point location = 2;
}
```

- `service`, `rpc`: dịch vụ và lời gọi **unary**
- `message`: mỗi trường có kiểu, tên và **số trường**; message lồng message
- Không có địa danh: `name` rỗng; trường không gán mang giá trị mặc định

Server: kế thừa lớp cơ sở sinh ra, trả kết quả qua StreamObserver

Java

```java
static final class RouteGuideService extends RouteGuideGrpc.RouteGuideImplBase {
    @Override
    public void getFeature(Point request, StreamObserver<Feature> responseObserver) {
        responseObserver.onNext(checkFeature(request));  // unary: gửi đúng một response
        responseObserver.onCompleted();  // rồi kết thúc lời gọi
    }
```

Kết quả chạy

```
gRPC server on port 9199, 8 features
GetFeature(20.9809, 105.7871) on grpc-default-executor-0
GetFeature(0.0000, 0.0000) on grpc-default-executor-0
```

Client: ManagedChannel và blocking stub cho lời gọi unary

Java

```
ManagedChannel channel = ManagedChannelBuilder.forAddress("127.0.0.1", 9199)
        .usePlaintext()  // HTTP/2 không mã hóa (h2c), chỉ để chạy cục bộ
        .build();
RouteGuideGrpc.RouteGuideBlockingStub stub = RouteGuideGrpc.newBlockingStub(channel);
for (Point point : new Point[] {Features.PTIT, Features.point(0, 0)}) {
    Feature feature = stub.getFeature(point);  // lời gọi unary qua blocking stub
```

Kết quả chạy

```
Found feature called "Học viện Công nghệ Bưu chính Viễn thông" at 20.9809, 105.7871
Found no feature at 0.0000, 0.0000
```

Đổi số trường làm client cũ đọc sai; chỉ thêm trường với số mới

Java

```
byte[] renumbered = PointRenumbered.newBuilder()
        .setLatitude(209_809_000).setLongitude(1_057_871_000).build().toByteArray();
Point point = Point.parseFrom(renumbered);  // client cũ đọc theo số trường cũ
byte[] extended = FeatureExtended.newBuilder()
        .setName("Học viện Công nghệ Bưu chính Viễn thông").setLocation(Features.PTIT).setCategory("giáo dục")
        .build().toByteArray();
Feature feature = Feature.parseFrom(extended);
```

Kết quả chạy

```
server đổi số trường: 105.7871, 20.9809, 0 unknown field(s)
server thêm trường 3: "Học viện Công nghệ Bưu chính Viễn thông" at 20.9809, 105.7871, 1 unknown field(s)
```

- Đổi số trường: client cũ đọc **sai**, vĩ độ thành 105,79, không có ngoại lệ
- Thêm trường với **số mới**: client cũ đọc đúng các trường cũ, bỏ qua trường lạ

Câu hỏi 1

Trong file .proto, khai báo int32 latitude = 1; Số 1 dùng để làm gì?

A. Quy định thứ tự in các trường khi in message ra màn hình.

B. Giới hạn trường latitude dài tối đa 1 byte.

C. Định danh trường trong dữ liệu mã hóa; tên latitude không đi trên mạng.

D. Số phiên bản của message Point.

Câu hỏi 1

Trong file .proto, khai báo int32 latitude = 1; Số 1 dùng để làm gì?

A. Quy định thứ tự in các trường khi in message ra màn hình.

B. Giới hạn trường latitude dài tối đa 1 byte.

C. Định danh trường trong dữ liệu mã hóa; tên latitude không đi trên mạng.

D. Số phiên bản của message Point.

Số trường được ghi vào tag của trường trong byte mã hóa; bên nhận dựa vào số này, không dựa vào tên, để biết byte thuộc trường nào.

Câu hỏi 2

gRPC chạy trên phiên bản HTTP nào, và điều đó mang lại gì cho nhiều lời gọi tới cùng một server?

A. HTTP/2: nhiều lời gọi chạy song song trên một kết nối nhờ multiplexing.

B. HTTP/1.1: mỗi lời gọi một kết nối riêng nên không phải chờ nhau.

C. HTTP/3: không bao giờ bị head-of-line blocking.

D. Không dùng HTTP: gRPC chạy thẳng trên UDP.

Câu hỏi 2

gRPC chạy trên phiên bản HTTP nào, và điều đó mang lại gì cho nhiều lời gọi tới cùng một server?

A. HTTP/2: nhiều lời gọi chạy song song trên một kết nối nhờ multiplexing.

B. HTTP/1.1: mỗi lời gọi một kết nối riêng nên không phải chờ nhau.

C. HTTP/3: không bao giờ bị head-of-line blocking.

D. Không dùng HTTP: gRPC chạy thẳng trên UDP.

gRPC dùng HTTP/2: mỗi lời gọi là một HTTP/2 stream, nhiều lời gọi chia nhau một kết nối TCP.

Tổng kết

1. **gRPC**: RPC trên HTTP/2; hợp đồng là **file .proto** viết bằng **Protocol Buffers**
2. **Sinh mã**: protoc tạo lớp message và stub; client gọi RPC **unary** qua **blocking stub**
3. Mã hóa theo số trường: message nhỏ hơn JSON; chỉ thêm trường mới, không đổi số trường đã dùng

Từ khóa cần nhớ gRPC · Protocol Buffers · file .proto · sinh mã · unary · blocking stub

Mở rộng

Chủ đề nâng cao

- Tương thích ngược và tương thích xuôi của Protocol Buffers
- gRPC-Web cho trình duyệt

Bài tập mở rộng

Sinh client từ route_guide.proto bằng một ngôn ngữ khác (Python hoặc Go) và gọi GetFeature tới server Java.

Tiếp theo

gRPC: streaming, deadline và mã trạng thái